

POLITECNICO
MILANO 1863

Ingegneria del Software

Ingegneria Informatica – Prof. San Pietro

Esercitazione:

Testing

My Contacts:
Giuseppe Sorrentino
giuseppe.sorrentino@polimi.it

Testing

Statement Coverage: insieme minimo di casi di test per garantire la copertura delle istruzioni

Edge Coverage: insieme minimo di casi di test per garantire la copertura delle decisioni

Path Coverage: insieme minimo di casi di test per garantire la copertura dei cammini

TdE 10/07/2023 – Esercizio 4

```
public static int test(int x, int y) {
1   int k = 0;
2   if(x>0) {
3   if(y>0) {
4       k += y;
5    } else {
6       k += -y;
7    }
8   }
9   if(x==10) {
10    k += 10;
11    }
12  return k;
13  }
```

Si consideri il seguente metodo statico:

Quesito a:
Si determini un insieme minimo di casi di test per garantire la copertura delle istruzioni (statement coverage).

Quesito b:
Si determini un insieme minimo di casi di test per garantire la copertura delle decisioni (edge coverage).

Quesito c:
Si determini un insieme minimo di casi di test per garantire la copertura dei cammini (path coverage).

TdE 10/07/2023 – Esercizio 4 – quesito a

```
public static int test(int x, int y) {
1   int k = 0;
2   if(x>0) {
3   if(y>0) {
4       k += y;
5    } else {
6        k += -y;
7    }
8   }
9   if(x==10) {
10    k += 10;
11    }
12  return k;
13  }
```

Lo scopo dello **statement coverage** è valutare quanti casi di test occorrono per testare **tutti** gli statement del codice.

TdE 10/07/2023 – Esercizio 4 – quesito a

```
public static int test(int x, int y) {
1   int k = 0;
2   if(x>0) {
3   if(y>0) {
4       k += y;
5    } else {
6        k += -y;
7    }
8   }
9   if(x==10) {
10    k += 10;
11    }
12  return k;
13  }
```

Lo scopo dello **statement coverage** è valutare quanti casi di test occorrono per testare **tutti** gli statement del codice.

Soluzione:

Servono almeno due casi di test al fine di entrare sia nel **ramo if** che nel **ramo else** dello statement condizionale all'interno del primo if. Ad esempio i due casi di test: (10,1) e (10,0)

TdE 10/07/2023 – Esercizio 4 – quesito b

```
public static int test(int x, int y) {
1   int k = 0;
2   if(x>0) {
3   if(y>0) {
4         k += y;
5     } else {
6           k += -y;
7     }
8   }
9   if(x==10) {
10     k += 10;
11     }
12  return k;
13  }
```

Lo scopo dell' **edge coverage** è valutare quali «brench» o «decisioni» vengono prese all'interno del codice.

TdE 10/07/2023 – Esercizio 4 – quesito b

```
public static int test(int x, int y) {
1   int k = 0;
2   if(x>0) {
3   if(y>0) {
4       k += y;
5    } else {
6        k += -y;
7    }
8   }
9   if(x==10) {
10    k += 10;
11    }
12  return k;
13  }
```

Lo scopo dell' **edge coverage** è valutare quali «brench» o «decisioni» vengono prese all'interno del codice.

Soluzione:

In aggiunta a quelli precedenti del quesito a, servono i casi di test necessari per entrare anche negli **else** dei due if più esterni. In questo caso specifico, come condizioni di test bastano (10,1) (10,0) e **(0,qualsiasi)**

TdE 10/07/2023 – Esercizio 4 – quesito c

```
public static int test(int x, int y) {
1   int k = 0;
2   if(x>0) {
3   if(y>0) {
4       k += y;
5    } else {
6        k += -y;
7    }
8   }
9   if(x==10) {
10    k += 10;
11    }
12  return k;
13  }
```

Lo scopo del **path coverage** è valutare quali sono tutti i possibili «cammini», ovvero gli insiemi di statement che possono essere eseguiti, data una certa combinazione di input.

Soluzione:

(10,1) -> {1,2,3,4,8,9,10,11,12}
(5,1) -> {1,2,3,4,12}
(10,0) -> {1,2,5,6,7,9,10,11,12}
(5,0) -> {1,2,5,6,7,12}
(0, any) -> {1,2}

TdE 15/02/2021

```
1 static int test(int x) {
2    int y = x % 3;
3    System.out.println(y);
4    while (x > 0 && y < 3) {
5       if (y == 1)
6           return y;
7        y--;
8    }
9    return 0;
10 }
```

Si determini un insieme minimo di test per ciascuno dei seguenti criteri di copertura. Qualora un criterio non sia soddisfacibile, calcolare la percentuale di copertura dell'insieme di test definito per quel criterio.

TdE 15/02/2021 – Copertura delle Istruzioni

```
1 static int test(int x) {
2    int y = x % 3;
3    System.out.println(y);
4    while (x > 0 && y < 3) {
5       if (y == 1)
6           return y;
7        y--;
8    }
9    return 0;
10 }
```

Si determini un insieme minimo di test per ciascuno dei seguenti criteri di copertura. Qualora un criterio non sia soddisfacibile, calcolare la percentuale di copertura dell'insieme di test definito per quel criterio.

Ci sono 2 Return **quindi** almeno 2 casi di test sono necessari!

TdE 15/02/2021 – Copertura delle Istruzioni

```
1 static int test(int x) {
2    int y = x % 3;
3    System.out.println(y);
4    while (x > 0 && y < 3) {
5       if (y == 1)
6           return y;
7        y--;
8    }
9    return 0;
10 }
```

Si determini un insieme minimo di test per ciascuno dei seguenti criteri di copertura. Qualora un criterio non sia soddisfacibile, calcolare la percentuale di copertura dell'insieme di test definito per quel criterio.

Ci sono 2 Return **quindi** almeno 2 casi di test sono necessari!

TdE 15/02/2021 – Copertura delle Istruzioni

```
1 static int test(int x) {
2    int y = x % 3;
3    System.out.println(y);
4    while (x > 0 && y < 3) {
5       if (y == 1)
6           return y;
7        y--;
8    }
9    return 0;
10 }
```

Si determini un insieme minimo di test per ciascuno dei seguenti criteri di copertura. Qualora un criterio non sia soddisfacibile, calcolare la percentuale di copertura dell'insieme di test definito per quel criterio.

Ci sono 2 Return **quindi** almeno 2 casi di test sono necessari!

X = 2 -> Statement: 1, 2, 3, 4, 5, 7, 4, 5, 6.

TdE 15/02/2021 – Copertura delle Istruzioni

```
1 static int test(int x) {
2    int y = x % 3;
3    System.out.println(y);
4    while (x > 0 && y < 3) {
5       if (y == 1)
6           return y;
7        y--;
8    }
9    return 0;
10 }
```

Si determini un insieme minimo di test per ciascuno dei seguenti criteri di copertura. Qualora un criterio non sia soddisfacibile, calcolare la percentuale di copertura dell'insieme di test definito per quel criterio.

Ci sono 2 Return **quindi** almeno 2 casi di test sono necessari!

X = 2 -> Statement: 1, 2, 3, 4, 5, 7, 4, 5, 6.
X = 0 -> Statement: 1, 2, 3, 4, 9

TdE 15/02/2021 – Copertura delle Decisioni

```
1 static int test(int x) {
2    int y = x % 3;
3    System.out.println(y);
4    while (x > 0 && y < 3) {
5       if (y == 1)
6            return y;
7        y--;
8    }
9    return 0;
10 }
```

Si determini un insieme minimo di test per ciascuno dei seguenti criteri di copertura. Qualora un criterio non sia soddisfacibile, calcolare la percentuale di copertura dell'insieme di test definito per quel criterio.

TdE 15/02/2021 – Copertura delle Decisioni

```
1 static int test(int x) {
2    int y = x % 3;
3    System.out.println(y);
4    while (x > 0 && y < 3) {
5       if (y == 1)
6            return y;
7        y--;
8    }
9    return 0;
10 }
```

Si determini un insieme minimo di test per ciascuno dei seguenti criteri di copertura. Qualora un criterio non sia soddisfacibile, calcolare la percentuale di copertura dell'insieme di test definito per quel criterio.

Devo esplorare TUTTI i rami, visibili e non, delle varie istruzioni Condizionali.

TdE 15/02/2021 – Copertura delle Decisioni

```
1 static int test(int x) {
2    int y = x % 3;
3    System.out.println(y);
4    while (x > 0 && y < 3) {
5       if (y == 1)
6           return y;
7        y--;
8    }
9    return 0;
10 }
```

Si determini un insieme minimo di test per ciascuno dei seguenti criteri di copertura. Qualora un criterio non sia soddisfacibile, calcolare la percentuale di copertura dell'insieme di test definito per quel criterio.

Devo esplorare TUTTI i rami, visibili e non, delle varie istruzioni Condizionali.

While -> Devo entrare nel loop, e anche saltarlo
IF -> Devo eseguire il return, **MA ANCHE SALTARLO**

TdE 15/02/2021 – Copertura delle Decisioni

```
1 static int test(int x) {
2    int y = x % 3;
3    System.out.println(y);
4    while (x > 0 && y < 3) {
5       if (y == 1)
6           return y;
7        y--;
8    }
9    return 0;
10 }
```

Si determini un insieme minimo di test per ciascuno dei seguenti criteri di copertura. Qualora un criterio non sia soddisfacibile, calcolare la percentuale di copertura dell'insieme di test definito per quel criterio.

Devo esplorare TUTTI i rami, visibili e non, delle varie istruzioni Condizionali.

While -> Devo entrare nel loop, e anche saltarlo
IF -> Devo eseguire il return, **MA ANCHE SALTARLO**



X = 0, X = 2 sono sufficienti,
come nel caso dello **statement coverage**

TdE 15/02/2021 – Copertura delle Decisioni

```
1 static int test(int x) {
2    int y = x % 3;
3    System.out.println(y);
4    while (x > 0 && y < 3) {
5       if (y == 1)
6           return y;
7        y--;
8    }
9    return 0;
10 }
```

Si determini un insieme minimo di test per ciascuno dei seguenti criteri di copertura. Qualora un criterio non sia soddisfacibile, calcolare la percentuale di copertura dell'insieme di test definito per quel criterio.

Devo esplorare TUTTI i rami, visibili e non, delle varie istruzioni Condizionali.

While -> Devo entrare nel loop, e anche saltarlo
IF -> Devo eseguire il return, **MA ANCHE SALTARLO**



X = 0, X = 2 sono sufficienti,
come nel caso dello **statement coverage**

TdE 15/02/2021 – Copertura delle Condizioni

```
1 static int test(int x) {
2    int y = x % 3;
3    System.out.println(y);
4    while (x > 0 && y < 3) {
5       if (y == 1)
6           return y;
7        y--;
8    }
9    return 0;
10 }
```

Si determini un insieme minimo di test per ciascuno dei seguenti criteri di copertura. Qualora un criterio non sia soddisfacibile, calcolare la percentuale di copertura dell'insieme di test definito per quel criterio.

Nella copertura di **Condizioni e Decisioni (punto C)** non devo solo assicurarmi che OGNI branch sia esplorato, ma anche che ogni **Condizione** atomica assuma, nei vari test, sia il valore di TRUE, che di FALSE.

TdE 15/02/2021 – Copertura delle Condizioni

```
1 static int test(int x) {
2    int y = x % 3;
3    System.out.println(y);
4    while (x > 0 && y < 3) {
5       if (y == 1)
6            return y;
7        y--;
8    }
9    return 0;
10 }
```

Si determini un insieme minimo di test per ciascuno dei seguenti criteri di copertura. Qualora un criterio non sia soddisfacibile, calcolare la percentuale di copertura dell'insieme di test definito per quel criterio.

Nella copertura di **Condizioni e Decisioni (punto C)** non devo solo assicurarmi che OGNI branch sia esplorato, ma anche che ogni **Condizione** atomica assuma, nei vari test, sia il valore di TRUE, che di FALSE.

Condizioni

C1: X > 0
C2: Y < 3
C3: Y == 1

TdE 15/02/2021 – Copertura delle Condizioni

```
1 static int test(int x) {
2    int y = x % 3;
3    System.out.println(y);
4    while (x > 0 && y < 3) {
5       if (y == 1)
6             return y;
7        y--;
8    }
9    return 0;
10 }
```

Si determini un insieme minimo di test per ciascuno dei seguenti criteri di copertura. Qualora un criterio non sia soddisfacibile, calcolare la percentuale di copertura dell'insieme di test definito per quel criterio.

Nella copertura di **Condizioni e Decisioni (punto C)** non devo solo assicurarmi che OGNI branch sia esplorato, ma anche che ogni **Condizione** atomica assuma, nei vari test, sia il valore di TRUE, che di FALSE almeno una volta.

Condizioni

C1: X > 0

C2: Y < 3

C3: Y == 1

Idealmente, occorre:

C1: T	C2: T	Per entrare nel Loop
C1: T	C2: F	Per avere False sulla C2
C1: F	C2:T	Per avere False su C1

TdE 15/02/2021 – Copertura delle Condizioni

```
1 static int test(int x) {
2    int y = x % 3;
3    System.out.println(y);
4    while (x > 0 && y < 3) {
5       if (y == 1)
6             return y;
7        y--;
8    }
9    return 0;
10 }
```

Si determini un insieme minimo di test per ciascuno dei seguenti criteri di copertura. Qualora un criterio non sia soddisfacibile, calcolare la percentuale di copertura dell'insieme di test definito per quel criterio.

Nella copertura di **Condizioni e Decisioni (punto C)** non devo solo assicurarmi che OGNI branch sia esplorato, ma anche che ogni **Condizione** atomica assuma, nei vari test, sia il valore di TRUE, che di FALSE almeno una volta.

Condizioni	Idealmente, occorre:		
C1: X > 0	C1: T	C2: T	Per entrare nel Loop
C2: Y < 3	C1: T	C2: F	Per avere False sulla C2
C3: Y == 1	C1: F	C2:T	Per avere False su C1

Per come è scritto il codice, se C1 è T, C2 è SEMPRE TRUE

TdE 15/02/2021 – Copertura delle Condizioni

```
1 static int test(int x) {
2    int y = x % 3;
3    System.out.println(y);
4    while (x > 0 && y < 3) {
5       if (y == 1)
6            return y;
7        y--;
8    }
9    return 0;
10 }
```

Si determini un insieme minimo di test per ciascuno dei seguenti criteri di copertura. Qualora un criterio non sia soddisfacibile, calcolare la percentuale di copertura dell'insieme di test definito per quel criterio.

Nella copertura di **Condizioni e Decisioni (punto C)** non devo solo assicurarmi che OGNI branch sia esplorato, ma anche che ogni **Condizione** atomica assuma, nei vari test, sia il valore di TRUE, che di FALSE almeno una volta.

Condizioni	Idealmente, occorre:		
C1: X > 0	C1: T	C2: T	Per entrare nel Loop
C2: Y < 3	C1: T	C2: F	Per avere False sulla C2
C3: Y == 1	C1: F	C2:T	Per avere False su C1

Per come è scritto il codice, se C1 è T, C2 è SEMPRE TRUE

Non è quindi possibile avere copertura completa, poiché solo 2/3 casi sono verificabili **-> 66.6%**

TdE 15/02/2021 – Errori di Programmazione?

```
1 static int test(int x) {
2    int y = x % 3;
3    System.out.println(y);
4    while (x > 0 && y < 3) {
5       if (y == 1)
6            return y;
7        y--;
8    }
9    return 0;
10 }
```

Si determini un insieme minimo di test per ciascuno dei seguenti criteri di copertura. Qualora un criterio non sia soddisfacibile, calcolare la percentuale di copertura dell'insieme di test definito per quel criterio.

TdE 15/02/2021 – Errori di Programmazione?

```
1 static int test(int x) {
2    int y = x % 3;
3    System.out.println(y);
4    while (x > 0 && y < 3) {
5       if (y == 1)
6           return y;
7        y--;
8    }
9    return 0;
10 }
```

Si determini un insieme minimo di test per ciascuno dei seguenti criteri di copertura. Qualora un criterio non sia soddisfacibile, calcolare la percentuale di copertura dell'insieme di test definito per quel criterio.

Per X = 3, il programma entra in un **loop infinito**

TdE 18/06/2022

```
1 public static int foo(int n) {
2 int x = n%2;
3 int y = n%3;
4 while (n != 0) {
5 if (x*y == 2 || n < 0) return x;
6 else n--;
7 }
8 return y;
9 }
```

Si determini un insieme minimo di test per ciascuno dei seguenti criteri di copertura. Qualora un criterio non sia soddisfacibile, calcolare la percentuale di copertura dell'insieme di test definito per quel criterio.

TdE 18/06/2022 – Copertura delle Istruzioni

```
1 public static int foo(int n) {
2 int x = n%2;
3 int y = n%3;
4 while (n != 0) {
5 if (x*y == 2 || n < 0) return x;
6 else n--;
7 }
8 return y;
9 }
```

Si determini un insieme minimo di test per ciascuno dei seguenti criteri di copertura. Qualora un criterio non sia soddisfacibile, calcolare la percentuale di copertura dell'insieme di test definito per quel criterio.

Quale insieme di test permette di esplorare TUTTE le istruzioni?

TdE 18/06/2022 – Copertura delle Istruzioni

```
1 public static int foo(int n) {
2 int x = n%2;
3 int y = n%3;
4 while (n != 0) {
5 if (x*y == 2 || n < 0) return x;
6 else n--;
7 }
8 return y;
9 }
```

Si determini un insieme minimo di test per ciascuno dei seguenti criteri di copertura. Qualora un criterio non sia soddisfacibile, calcolare la percentuale di copertura dell'insieme di test definito per quel criterio.

Quale insieme di test permette di esplorare TUTTE le istruzioni?

N=2 -> Statement: 1, 2, 3, 4, 6, 4, 6, 8
N=5 -> Statement: 1, 2, 3, 4, 5

TdE 18/06/2022 – Copertura delle Decisioni

```
1 public static int foo(int n) {
2 int x = n%2;
3 int y = n%3;
4 while (n != 0) {
5 if (x*y == 2 || n < 0) return x;
6 else n--;
7 }
8 return y;
9 }
```

Si determini un insieme minimo di test per ciascuno dei seguenti criteri di copertura. Qualora un criterio non sia soddisfacibile, calcolare la percentuale di copertura dell'insieme di test definito per quel criterio.

Quale insieme di test permette di esplorare TUTTE le decisioni (ovvero avere T e F per ogni condizione?

TdE 18/06/2022 – Copertura delle Decisioni

```
1 public static int foo(int n) {
2 int x = n%2;
3 int y = n%3;
4 while (n != 0) {
5 if (x*y == 2 || n < 0) return x;
6 else n--;
7 }
8 return y;
9 }
```

Si determini un insieme minimo di test per ciascuno dei seguenti criteri di copertura. Qualora un criterio non sia soddisfacibile, calcolare la percentuale di copertura dell'insieme di test definito per quel criterio.

Quale insieme di test permette di esplorare TUTTE le decisioni (ovvero avere T e F per ogni condizione?

N=2
While: T, T, F
IF/Else: F, F, F

N=5
While: T
IF/Else: T

TdE 18/06/2022 – Copertura delle Condizioni

```
1 public static int foo(int n) {
2 int x = n%2;
3 int y = n%3;
4 while (n != 0) {
5 if (x*y == 2 || n < 0) return x;
6 else n--;
7 }
8 return y;
9 }
```

Si determini un insieme minimo di test per ciascuno dei seguenti criteri di copertura. Qualora un criterio non sia soddisfacibile, calcolare la percentuale di copertura dell'insieme di test definito per quel criterio.

Quale insieme di test permette di esplorare TUTTE le decisioni **E condizioni?**

TdE 18/06/2022 – Copertura delle Condizioni

```
1 public static int foo(int n) {
2 int x = n%2;
3 int y = n%3;
4 while (n != 0) {
5 if (x*y == 2 || n < 0) return x;
6 else n--;
7 }
8 return y;
9 }
```

Si determini un insieme minimo di test per ciascuno dei seguenti criteri di copertura. Qualora un criterio non sia soddisfacibile, calcolare la percentuale di copertura dell'insieme di test definito per quel criterio.

Quale insieme di test permette di esplorare TUTTE le decisioni **E condizioni?**

N=2

N!=0 assume sia True che False

N=5

N!=0 assume TRUE

X*Y==2 assume TRUE

N<0 assume FALSE

Cosa manca?

TdE 18/06/2022 – Copertura delle Condizioni

```
1 public static int foo(int n) {
2 int x = n%2;
3 int y = n%3;
4 while (n != 0) {
5 if (x*y == 2 || n < 0) return x;
6 else n--;
7 }
8 return y;
9 }
```

Si determini un insieme minimo di test per ciascuno dei seguenti criteri di copertura. Qualora un criterio non sia soddisfacibile, calcolare la percentuale di copertura dell'insieme di test definito per quel criterio.

Quale insieme di test permette di esplorare TUTTE le decisioni **E condizioni?**

N=2
N!=0 assume sia True che False

N=5
N!=0 assume TRUE
X*Y==2 assume TRUE
N<0 assume FALSE

Cosa manca? Manca un caso che costringa N<0 ad essere TRUE con X*Y==2 pari a FALSE

TdE 18/06/2022 – Copertura delle Condizioni

```
1 public static int foo(int n) {
2 int x = n%2;
3 int y = n%3;
4 while (n != 0) {
5 if (x*y == 2 || n < 0) return x;
6 else n--;
7 }
8 return y;
9 }
```

Si determini un insieme minimo di test per ciascuno dei seguenti criteri di copertura. Qualora un criterio non sia soddisfacibile, calcolare la percentuale di copertura dell'insieme di test definito per quel criterio.

Quale insieme di test permette di esplorare TUTTE le decisioni **E condizioni?**

N=2
N!=0 assume sia True che False

N=5
N!=0 assume TRUE
X*Y==2 assume TRUE
N<0 assume FALSE

N=1
N!=0 assume sia True che False
X*Y==2 FALSE,
N < 0 prima FALSE, poi TRUE